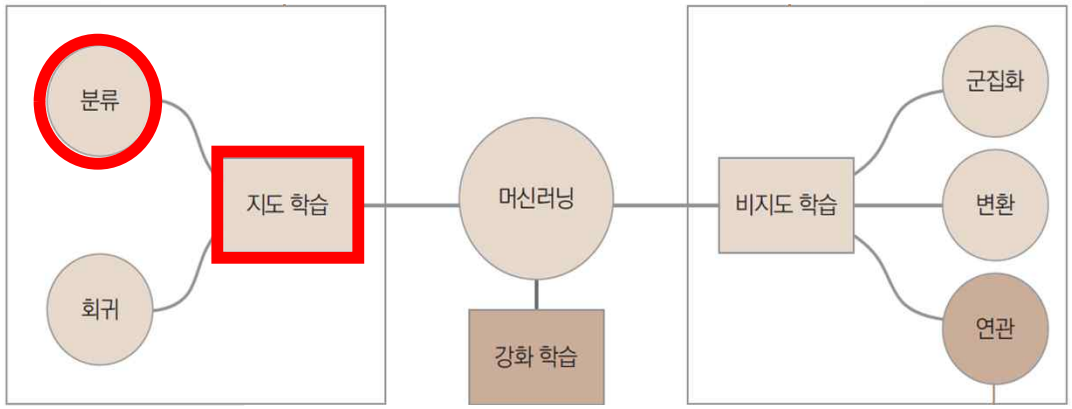
머신러닝 프로그래밍
Machine Learning Programming

Lecture 4 : 로지스틱회귀

충북대학교
CHUNGBUK NATIONAL UNIVERSITY
충북대학교
산업인공지능연구센터

머신러닝 기법 : **지도학습**, 비지도학습, 강화학습



- 회귀 : 연속된 값을 측정하는 문제
  (ex) 중고차 가격 예측
- **분류 : 범주형 값을 측정하는 문제**
  (ex) 마케팅 문자를 보냈을 때 구매할 것 같은 고객을 예측하는 문제는 Yes/No 두 가지 범주로 표현

- 군집화(clustering) : 비슷한 데이터끼리 묶어주는 방법
  (ex) 소비 패턴이 유사한 카드 고객을 묶음.

- 변환(transformation) : 목적에 따라 데이터를 다른 형태로 바꿈.
  (ex) 변수 100개의 특성을 최대한 살려 변수 2개로 압축

- 연관(association) : 일명 장바구니 분석이라고도 하며, 유사한 구매 목록을 가진 고객끼리 비교하여, 서로의 장바구니에 담기지 않은 새로운 아이템을 추천을 하는 목적 등으로 사용.

분류(Classification) vs 회귀(Regression)

구분	분류 (Classification)	회귀 (Regression)
목표	데이터가 어떤 카테고리(범주)에 속하는지 예측 Which one ?	데이터의 연속적인 수치(값)를 예측 How much ?
결과값	이산적	연속적
예시	스팸 메일 차단, MNIST(0~9 숫자판별)	주가예측, 기온예측
평가방법	정밀도, 재현율, ROC	MSE(평균제곱오차), R^2 *Score*
문제확장	다중분류(3개이상)	다중회귀 (입력 특성이 여러 개)
심화분석	오류분석(혼동 행렬 시각화)	잔차분석(오차의 패턴확인)
복합예측	다중 레이블 / 다중출력	다변량 회귀 (출력값이 여러 개인 경우)
핵심 모델	로지스틱 회귀(4주차), 의사결정트리(5주차) K-최근접이웃(6주차), 서포트벡터벡신(7주차)	선형회귀(9주차), 앙상블(10주차)

✓ 로지스틱 회귀 : 작동원리가 선형회귀와 거의 유사, but 시그모이드(sigmoid) 함수에 통과시켜 0과 1사이의 확률값으로 변환하하여 분류하기 때문에 분류로 봄

로지스틱 회귀

오즈 (odds)

- 성공(y = 1) 확률이 실패(y = 0) 확률에 비해 몇 배 더 높은가를 나타냄

$$\text{odds} = \frac{p(y=1|x)}{1-p(y=1|x)}$$

로짓 변환 (logit)

- 오즈에 로그를 취한 함수 형태
- 입력값 (p)의 범위가 [0, 1] 일 때, [- ∞, + ∞] 를 출력함

$$\text{logit}(p) = \log(\text{odds}) = \log\frac{p(y=1|x)}{1-p(y=1|x)}$$

로지스틱 함수 (Logistic Function)

- **로짓 변환의역함수.** 로짓 변환을 선형 결합과 같다고 두면,

$$\text{logit}(p) = \log(\text{odds}) = \log\frac{p(y=1|x)}{1-p(y=1|x)} = w_0 + w_1x_1 + \cdots + w_Dx_D = w^TX$$

$$p(y=1|x) = \frac{e^{w^TX}}{1+e^{w^TX}} = \frac{1}{1+e^{-w^TX}}$$

다중클래스 분류의 개념

- 다중클래스 분류(multi-class classification) : 2개 이상의 클래스를 가진 y 값에 대한 분류

다중클래스와 다중레이블

분류	다중클래스(multi-class) 분류	다중레이블(multi-label) 분류
작업	2개 이상의 클래스를 가진 분류 작업	상호 배타적이지 않은 속성 예측
중복 선택	중복 선택 불가능 →[1 0 0] 가능, [1 1 0] 불가	중복 선택 가능 →[1 1 0] 가능
예	과일 사진 분류 : 오렌지, 사과, 배	신문기사 분류 : 운동선수-연예인 결혼 기사 → 스포츠/연예 면

분류 접근

- One-vs-All : m개의 클래스가 존재할 때 각 클래스마다 분류기(classifier)를 생성하여 분류
  - One-vs-Rest라고도 부름
  - 대표적으로 소프트맥스 분류(softmax classification)
- One-vs-One : m개의 클래스가 있다면, 이 클래스의 분류기를 하나의 클래스로 하고 나머지 클래스의 분류기들을 만들어 최종적으로 각 분류기들의 결과를 투표로 결정
  - 총 $\frac{m(m-1)}{2}$ 개만큼의 분류기를 생성
  - 분류기가 많아질수록 정확도 높아지지만 비용도 증가

소프트맥스 분류

소프트맥스 함수

- 시그모이드 함수로 다중클래스 분류 문제 다룰 수 있음
  - 각각의 클래스에 속하는지 속하지 않는지 이진분류기 m개를 생성한 후, 가장 높은 확률이 나오는 클래스를 선택
  - 분류기 번호 m에 대해 $h_m(x;\theta)$ 로 표현
  - 그러나 $h_m(x;\theta)$ 확률의 합이 1 이상이 된다는 문제 발생
  - 문제 해결 방법은 모든 클래스들의 발생 확률을 1로 정규화

로지스틱 회귀_다중클래스 분류와 소프트맥스 분류

- 소프트맥스 함수(softmax function) : 다중클래스 분류에서 여러 선형회귀의 출력 결과를 정규화하여 합이 1이 되도록 만드는 함수

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}} \qquad \text{for } j = 1, 2, 3, \cdots, K$$

표 10-2 소프트맥스 함수 값 정리

z_j	e^{z_j}	$\frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$
2	7.389	0.609
1	2.718	0.224
−1	0.367	0.030
0.5	1.648	0.135

$$\sum_{j=1}^{K} \sigma(z)_j = \sum_{j=1}^{K} P_j = 1$$

지수 함수 그래프 시각화 ($y = e^x$)

$$e^{-2} > \frac{1}{e^2} \approx \frac{1}{7.389} \approx 0.135$$

가설함수 H(X)의 역할

확률적 예측

입력 데이터 x를 기반으로 특정 클래스에 속할 **확률 (Probability)**을 예측하는 함수입니다. 모델의 최종 판단 근거가 됩니다.

최종 출력층 처리

신경망의 가장 마지막 단계인 **출력층(Output Layer)**에 위치하여, 가중치 θ와 입력의 선형 결합을 확률로 변환합니다.

로지스틱 회귀_다중클래스 분류와 소프트맥스 분류

SIGMOID VS SOFTMAX 비교 분석

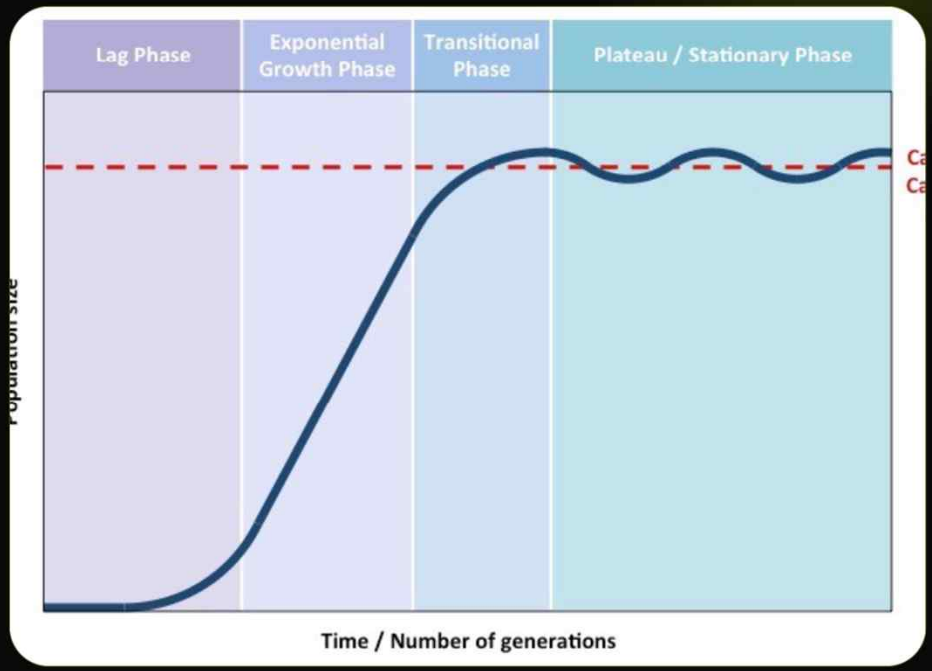
구분	시그모이드 (Sigmoid)	소프트맥스 (Softmax)
분류 목적	이진 분류 (Binary)	다중 클래스 분류 (Multiclass)
출력 개수	1개 (단일 확률값)	K개 (확률 벡터)
결과값의 합	해당 없음 (0~1 사이)	항상 1 (100%)
주요 용도	Logistic Regression	Deep Learning / CNN / RNN

시그모이드의 확률 매핑

시그모이드 함수는 선형 결합 값 **z = θᵀx**를 0과 1 사이의 값으로 압축(Squeezing)합니다.

$$h\left(x\right) = \frac{1}{1 + e^{-\theta^T x}}$$

결정 경계(Decision Boundary) **0.5**를 기준으로 양성(1) 또는 음성(0) 클래스를 판단합니다.



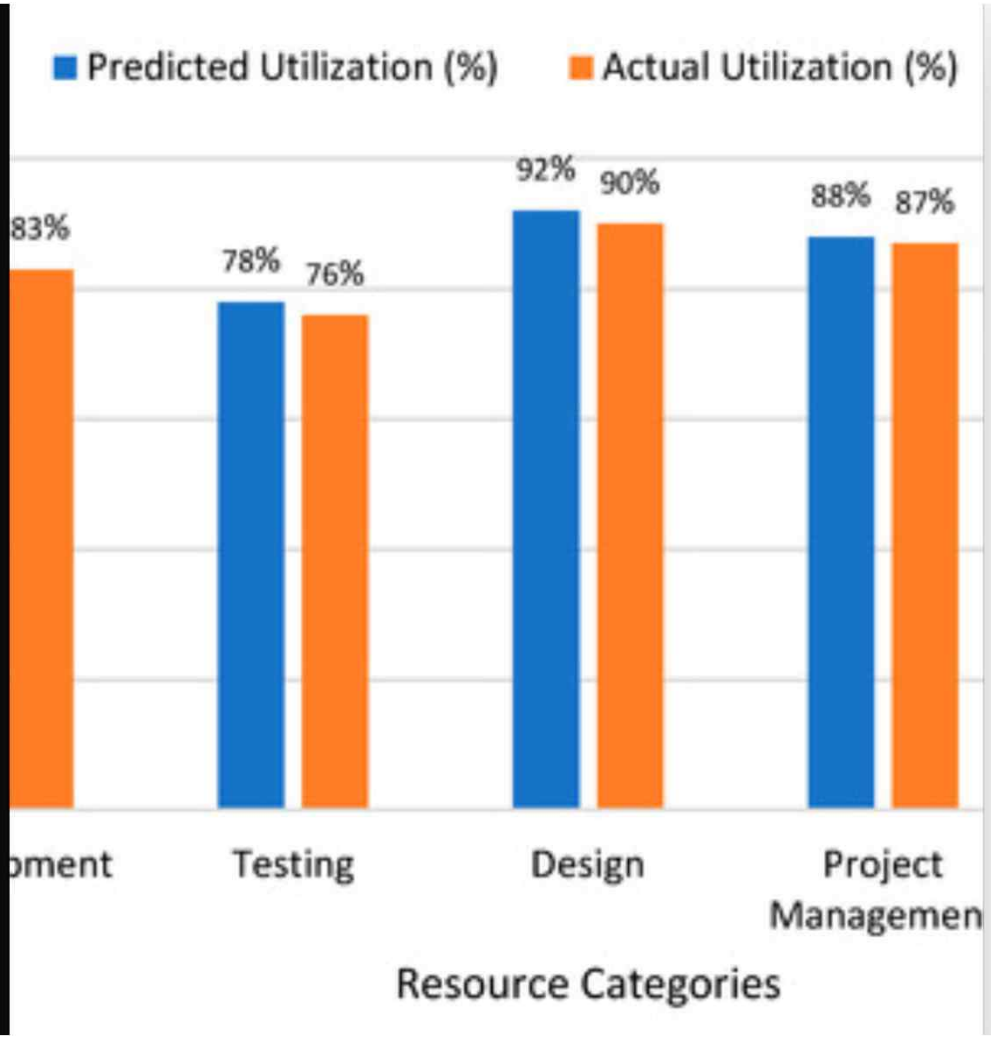
로지스틱 회귀_다중클래스 분류와 소프트맥스 분류

소프트맥스의 확률 분포

다중 클래스 환경에서 각 클래스의 점수를 지수화하여 상대적 비중을 계산합니다.

$$h_j\left(x\right) = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$

모든 클래스의 확률 합이 **1**이 되도록 정규화(Normalization)를 수행하여 직관적인 확률 해석이 가능합니다.



로지스틱 회귀_다중클래스 분류와 소프트맥스 분류

4-1. softmax.py

```python
import numpy as np

def softmax(values):  1개의 사용 위치

    # 1. 각 요소에 지수 함수(e^x)를 적용합니다.
    array_values = np.exp(values)

    # 2. 각 요소를 전체 합으로 나누어 결과의 총합이 1이 되도록 정규화합니다.
    return array_values / np.sum(array_values)

values = [-2, -1, -5, 0.5]

# 소프트맥스 함수 실행
y = softmax(values)

print("1. 입력값 (Values):", values)
print("2. 소프트맥스 결과 (y):")
for i, prob in enumerate(y):
    print(f"   - 클래스 {i}의 확률: {prob:.8f} ({prob * 100:.2f}%)")

# 결과의 합이 1인지 확인
total_sum = y.sum()
print("\n3. 결과값의 총합 (y.sum()):", total_sum)

# 결과 해석
max_index = np.argmax(y)
print(f"\n 가장 높은 확률을 가진 인덱스는 {max_index}번이며, 값은 {values[max_index]}입니다.")
```

소프트맥스 함수로 구현하는 소프트맥스 분류

- 오즈비에 logit 함수를 붙여 최종적으로 구한 가중치 값

$$\frac{P_j}{1-P_j} \Rightarrow logit(P_j) = \log_e\left(\frac{P_j}{1-P_j}\right) = z = \theta^T X$$

- 기존의 오즈비는 이진분류이지만, 다중클래스 분류는 j 번째 대상에 대한 전체 대비 비율을 나타냄

$$\frac{P_j}{P_K} \Rightarrow \log \ddot{y}(P_j) = \log_e\left(\frac{P_j}{P_K}\right) = z_j = X^T\theta_j$$

$$\frac{P_j}{P_K} = e^{z_j}$$

로지스틱 회귀_다중클래스 분류와 소프트맥스 분류

- 다중클래스 분류에서는 j개의 클래스가 있다면 클래스의 개수만큼 가중치에 대한 벡터를 구함
- 피쳐 벡터와 각 클래스별로 존재하는 가중치 행렬의 선형결합을 z로 나타냄

$$\sum_{j=1}^{K}\frac{P_j}{P_K}=\sum_{j=1}^{K}e^{z_j}\Rightarrow\frac{1}{P_K}\sum_{j=1}^{K}P_j=\sum_{j=1}^{K}e^{z_j}\Rightarrow$$

$$\frac{1}{P_K}\times 1=\sum_{j=1}^{K}e^{z_j}\Rightarrow\left(\because\sum_{j=1}^{K}P_j=1\right)$$

$$P_k=\frac{1}{\sum_{j=1}^{K}e^{z_j}}$$

로지스틱 회귀_다중클래스 분류와 소프트맥스 분류

$$\frac{P_j}{P_k} = e^{z_j} \Rightarrow \frac{P_j}{\sum_{j=1}^{K} e^{z_j}} = e^{z_j} \Rightarrow \because P_K = \frac{1}{\sum_{j=1}^{K} e^{z_j}}$$

$$P_j = \frac{e^{z_j}}{\sum_{j=1}^{K} e^{z_j}}$$

$$P_j = \frac{e^{z_j}}{\sum_{j=1}^{K} e^{z_j}} = \frac{e^{z_j}}{\sum_{j=1}^{K} e^{x^T\theta_j}} \quad \because z = X^T\theta_j$$

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_3 \end{bmatrix} = \begin{bmatrix} w_{10} & w_{11} & \cdots & w_{1i} \\ w_{20} & w_{21} & \cdots & w_{2i} \\ \vdots & \vdots & \ddots & \vdots \\ w_{j0} & w_{j1} & \cdots & w_{ji} \end{bmatrix}$$

소프트맥스 함수로 학습하기

- 오른쪽 수식에서 θ를 학습
  즉 각 클래스마다 적절한 θ_j를 찾기

$$P_j = \frac{e^{x^T\theta_j}}{\sum_{j=1}^{K} e^{X^T\theta_j}}$$

- 각 가설함수는 각 클래스와 발생확률로 표현 가능

$$h_\theta(x) = \begin{bmatrix} P(y=1\,|\,x;\theta) \\ P(y=2\,|\,x;\theta) \\ \vdots \\ P(y=K\,|\,x;\theta) \end{bmatrix} = \frac{1}{\sum_{j=1}^{K}\exp(\theta^{(j)T}x)} \begin{bmatrix} \exp(\theta^{(1)T}x) \\ \exp(\theta^{(2)T}x) \\ \vdots \\ \exp(\theta^{(K)T}x) \end{bmatrix}$$

- 최대우도추정법(Maximum Likelihood Estation, MLE)을 사용해서 P_j 확률을 최대화하는 θ를 찾기

$$\arg\max_{\theta} \prod_{i=1}^{m} P(y^{(i)}\,|\,x^{(i)};\theta)$$

로지스틱 회귀_다중클래스 분류와 소프트맥스 분류

- 위 수식을 손실(loss)로 생각하여 수식 L로 표현하고 해당 값을 최대화하는 방향으로 정리

$$L = \prod_{i=1}^{m} P(y^{(i)} | x^{(i)}; \theta) = \prod_{i=1}^{m} \prod_{j=1}^{K} p_j^{(i)^{v_{ij}}} \Rightarrow$$

$$-\log L = -\log \prod_{i=1}^{m} \prod_{j=1}^{K} p_j^{(i)^{v_{ij}}} = -\sum_{i=1}^{m} \sum_{j=1}^{K} v_{ij} \log p_j^{(i)}$$

$$\text{where } v_{ij} \begin{cases} 1 & \text{if } y^{(i)} \text{is label } j \\ 0 & \text{if } y^{(i)} \text{is NOT label } j \end{cases}$$

$$l = -\sum_{i=1}^{m} \sum_{j=1}^{K} v_{ij} \log p_j^{(i)} \qquad where p_j^{(i)} = \frac{e^{z_j^{(i)}}}{\sum_{j=1}^{K} e^{z_j^{(i)}}}$$

- 최종적으로 $\frac{\partial l}{\partial z} \frac{\partial z}{\partial \theta}$ 을 구하고자 함

mnist 데이터셋의 이해

- 손글씨를 숫자로 인식하는 이미지 분류 문제

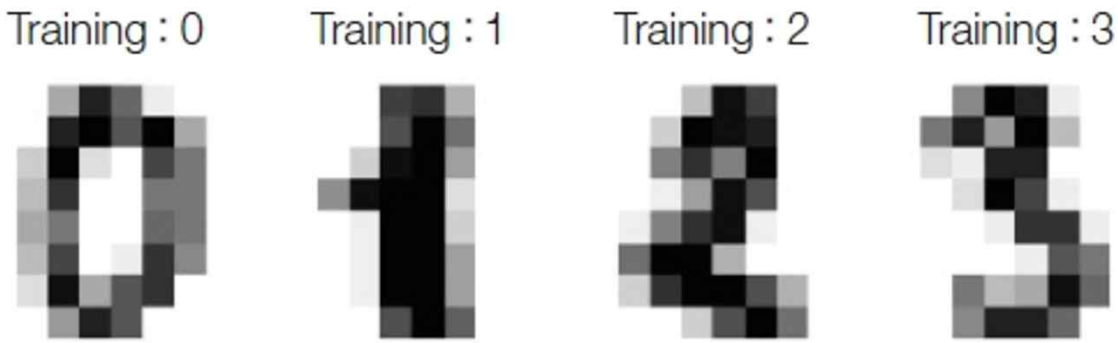


그림 10-1 사이킷런의 mnist 데이터셋 예제

- 컴퓨터는 이미지를 일종의 숫자로 변환하여 인식
  - 이미지를 일종의 점(dot)으로 생각하면 m×n만큼의 공간이 존재하고, 그 공간 안에서 색깔이 진할수록 높은 값, 색깔이 옅을수록 낮은 값을 가짐

로지스틱 회귀_다중클래스 분류를 코드로 구현

4-2. softmax_graph.py

```python
import numpy as np
import matplotlib.pyplot as plt

def softmax(values):  1개의 사용 위치
    # 1. 지수 함수 적용 (음수도 양수로 변환)
    array_values = np.exp(values)
    # 2. 총합이 1이 되도록 정규화
    return array_values / np.sum(array_values)

# --- 입력 데이터  --
values = [-2, -1, -5, 0.5]
labels = [f'Class {i}\n({v})' for i, v in enumerate(values)]

# 소프트맥스 계산
y = softmax(values)

# --- 그래프 그리기 ---
plt.figure(figsize=(10, 6))

# 막대 그래프 생성
bars = plt.bar(labels, y, color=['#4f46e5', '#818cf8', '#c7d2fe', '#f43f5e'])

# 그래프 위에 확률 값 표시 (%)
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height + 0.01,
             s: f'{height*100:.1f}%', ha='center', va='bottom', fontweight='bold')

# 그래프 꾸미기
plt.title( label: "Softmax Output: Probability Distribution", fontsize=15, pad=20)
plt.xlabel( xlabel: "Classes (Input Score)", fontsize=12)
plt.ylabel( ylabel: "Probability", fontsize=12)
plt.ylim( *args: 0, 1.1)  # 확률은 0~1 사이이므로 여유 있게 설정
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# 텍스트 결과 출력
print("입력 점수:", values)
print("확률 변환:", [f"{prob*100:.2f}%" for prob in y])
```

로지스틱 회귀_다중클래스 분류를 코드로 구현

4-2. softmax_graph_2.py

```python
import numpy as np
import matplotlib.pyplot as plt

def softmax(values):  1개의 사용 위치
    array_values = np.exp(values)
    return array_values / np.sum(array_values)

# --- 데이터 준비 ---
values = [-2, -1, -5, 0.5]
y = softmax(values)
labels = [f'Class {i} ({v})' for i, v in enumerate(values)]
colors = ['#4f46e5', '#818cf8', '#c7d2fe', '#f43f5e']

# --- 그래프 그리기 ---
fig, ax = plt.subplots(figsize=(12, 4))

# 단일 누적 가로 막대 그래프 생성
left_pos = 0  # 누적 시작 지점
for i in range(len(y)):
    ax.barh( y: 'Softmax Total (1.0)', y[i], left=left_pos, color=colors[i], label=labels[i], edgecolor='white', height=0.6)

    # 막대 중앙에 확률 값(%) 표시 (비중이 너무 작으면 텍스트 생략)
    if y[i] > 0.03:
        ax.text(left_pos + y[i] / 2,  y: 0,  s: f'{y[i] * 100:.1f}%',
                va='center', ha='center', color='white' if i != 2 else 'black', fontweight='bold', fontsize=11)

    left_pos += y[i]

# 그래프 꾸미기
ax.set_title( label: "Softmax Output: 100% Stacked Visualization", fontsize=16, pad=20)
ax.set_xlim( left: 0,  right: 1)  # 전체 길이를 딱 1로 고정
ax.set_xticks([0, 0.25, 0.5, 0.75, 1.0])
ax.set_xticklabels(['0', '0.25', '0.5', '0.75', '1.0 (Total)'])
ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.15), ncol=4, frameon=False)

# 불필요한 테두리 제거
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)

plt.tight_layout()
plt.show()

# 검증용 출력
print("각 클래스 확률:", [f"{prob:.4f}" for prob in y])
print("확률 총합:", np.sum(y))
```

로지스틱 회귀_다중클래스 분류를 코드로 구현

4-3. logistic.py

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.linear_model import LogisticRegression

# 1. 데이터 준비 (0, 1, 2 세 가지 숫자만 사용)
digits = datasets.load_digits()
X, y = digits.data, digits.target
mask = np.isin(y, test_elements: [0, 1, 2])
X_sub, y_sub = X[mask], y[mask]

# --- [실습 1] 시그모이드 (Sigmoid): "이것은 0입니까?" ---# 0이면 1(True), 1이나 2이면 0(False)으로
y_binary = (y_sub == 0).astype(int)
model_sigmoid = LogisticRegression(max_iter=10000).fit(X_sub, y_binary)
# 첫 번째 샘플에 대한 확률 ( [0이 아닐 확률, 0일 확률] )
prob_sigmoid = model_sigmoid.predict_proba(X_sub[:1])[0]

# --- [실습 2] 소프트맥스 (Softmax): "0, 1, 2 중 누구입니까?" ---
model_softmax = LogisticRegression(max_iter=10000).fit(X_sub, y_sub)
# 첫 번째 샘플에 대한 확률 ( [0일 확률, 1일 확률, 2일 확률] )
prob_softmax = model_softmax.predict_proba(X_sub[:1])[0]

# 2. 결과 출력 및 시각화
print(f"=== 확률 결과 비교 ===")
print(f"Sigmoid Output (Is it 0?): {prob_sigmoid}")
print(f"Softmax Output (0, 1, or 2?): {prob_softmax}")

plt.figure(figsize=(12, 5))

# 왼쪽: 시그모이드 결과 (이진 분류 - 선택지가 2개)
# 질문: "0인가요?" -> 대답: "네(Is 0)" 혹은 "아니오(Not 0)"
plt.subplot( *args: 1, 2, 1)
plt.bar( x: ['Not 0', 'Is 0'], prob_sigmoid, color=['lightgray', 'royalblue'], alpha=0.8)
plt.title( label: "Sigmoid (Binary Classification)", fontsize=14)
plt.ylabel("Probability")
plt.ylim( *args: 0, 1.1)
for i, v in enumerate(prob_sigmoid):
    plt.text(i, v + 0.02, s: f"{v:.2f}", ha='center')

plt.subplot( *args: 1, 2, 2)
plt.bar( x: ['Digit 0', 'Digit 1', 'Digit 2'], prob_softmax, color=['tomato', 'mediumseagreen', 'orange'], alpha=0.8)
plt.title( label: "Softmax (Multiclass Classification)", fontsize=14)
plt.ylabel("Probability")
plt.ylim( *args: 0, 1.1)
for i, v in enumerate(prob_softmax):
    plt.text(i, v + 0.02, s: f"{v:.2f}", ha='center')

plt.tight_layout()
plt.show()
```

데이터 불러오기

- datasets 모듈을 호출
- load_digits 함수로 딕셔너리 타입 데이터를 불러온다

In [1]:	`from sklearn import datasets`
	`digit_dataset = datasets.load_digits()`
	`digit_dataset.keys()`

Out [1]:	`dict_keys(['data', 'target', 'frame',` `'feature_names', 'target_names', 'images', 'DESCR'])`

로지스틱 회귀_다중클래스 분류를 코드로 구현

In [2]:	`digit_dataset["images"].shape`
Out [2]:	`(1797, 8, 8)`
In [3]:	`digit_dataset["target"][0]`
Out [3]:	`0`
In [4]:	`digit_dataset["images"][0]`

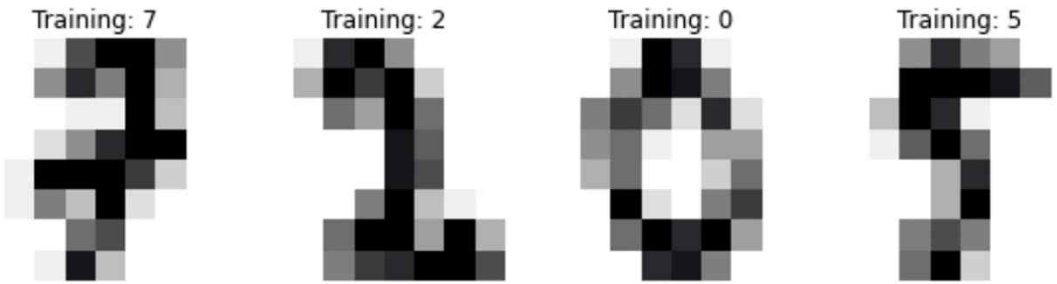
Out [4]:	`array([[ 0., 0., 5., 13., 9., 1., 0., 0.],`
	`[ 0., 0., 13., 15., 10., 15., 5., 0.],`
	`[ 0., 3., 15., 2., 0., 11., 8., 0.],`
	`[ 0., 4., 12., 0., 0., 8., 8., 0.],`
	`[ 0., 5., 8., 0., 0., 9., 8., 0.],`
	`[ 0., 4., 11., 0., 1., 12., 7., 0.],`
	`[ 0., 2., 14., 5., 10., 12., 0., 0.],`
	`[ 0., 0., 6., 13., 10., 0., 0., 0.]])`

로지스틱 회귀_다중클래스 분류를 코드로 구현

In [5]:

```
import matplotlib.pyplot as plt
from random import randint
_, axes = plt.subplots(nrows=1, ncols=4, figsize=(10, 3))
# (1)

for ax in axes: # (2)
    num = randint(1, 1000) # (3)
    image = digit_dataset["images"][num]
    label = digit_dataset["target"][num]
    ax.set_axis_off()
    ax.imshow(image, cmap=plt.cm.gray_r,
interpolation='nearest') # (4)
    ax.set_title('Training: %i' % label)
```

Out [5]:



[TIP] 결과값에 색을 지정하는 요소(property)인 plt.cm.gray_r을 변경하면
좀 더 다양한 형태로 값 표현이 가능하다

- 데이터가 8×8 행렬이므로 2D 이미지로 표현되었지만 다음 코드와 같이 총 64개의 피쳐(feature)를 가진 하나의 데이터로 받을 수 있음

In [6]:	`digit_dataset["data"][0].shape`
Out [6]:	`(64,)`

데이터 분류하기

- 데이터를 훈련 데이터셋과 테스트 데이터셋으로 구분

In [7]:
```
from sklearn.model_selection import train_test_split

X = digit_dataset["data"] # (1)
y = digit_dataset["target"] # (1)
X_train, X_test, y_train, y_test = train_test_split(X,
y) # (2)
```

모델 생성하기

- ovr : 클래스 모드를 모두 이진모델로 만들어 학습
- multinomial : 소프트맥스 함수를 사용하여 계산하는 방식. 경사하강법의 매개변수 solver를 sag으로 변경

In [8]:
```
from sklearn.linear_model import LogisticRegression

logreg_ovr = LogisticRegression(multi_class="ovr")
logreg_softmax =
LogisticRegression(multi_class="multinomial",
solver="sag")

logreg_ovr.fit(X_train, y_train)
logreg_softmax.fit(X_train, y_train)
```

Out [8]:
```
LogisticRegression(multi_class='multinomial',
solver='sag')
```

성능 측정하기

- 일반적으로 다중클래스 분류도 기존 혼동행렬을 사용
- 각 클래스 대비 예측한 값을 행렬 형태로 표현

In [9]:
```
from sklearn.metrics import confusion_matrix
y_pred = logreg_ovr.predict(X_test).copy()
y_true = y_test.copy()
confusion_matrix(y_true, y_pred)
```

Out [9]:
```
array([[47, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [ 0, 49, 1, 0, 0, 0, 0, 0, 0, 0],
       [ 0, 0, 49, 2, 0, 0, 0, 0, 0, 0],
       [ 0, 0, 0, 37, 0, 1, 0, 0, 1, 0],
       [ 0, 0, 0, 0, 41, 0, 0, 0, 0, 0],
       [ 0, 1, 0, 0, 1, 41, 1, 0, 0, 1],
       [ 0, 0, 0, 0, 2, 0, 36, 0, 0, 0],
       [ 0, 0, 0, 0, 1, 0, 0, 41, 0, 1],
       [ 0, 0, 0, 0, 0, 0, 0, 0, 44, 0],
       [ 1, 0, 0, 0, 0, 0, 0, 0, 1, 50]], dtype=int64)
```

로지스틱 회귀_다중클래스 분류를 코드로 구현

- 라벨별로 분류 성능을 수치화하여 표시

In [10]:
```
from sklearn.metrics import classification_report
print(classification_report(y_true, y_pred))
```

Out [10]:
```
              precision recall f1-score support
           0       0.98     1.00     0.99      47
           1       0.98     0.98     0.98      50
           2       0.98     0.96     0.97      51
           3       0.95     0.95     0.95      39
           4       0.91     1.00     0.95      41
           5       0.98     0.91     0.94      45
           6       0.97     0.95     0.96      38
           7       1.00     0.95     0.98      43
           8       0.96     1.00     0.98      44
           9       0.96     0.96     0.96      52

    accuracy       0.97      450
   macro avg       0.97     0.97     0.97     450
weighted avg       0.97     0.97     0.97     450
```

로지스틱 회귀_다중클래스 분류를 코드로 구현

- micro를 선택하면 전체 평균값, macro를 선택하면 각 라벨별 결과의 합에 대한 평균을 나타냄

In [11]:	`result = confusion_matrix(y_true, y_pred)` `result.diagonal().sum() / result.sum(axis=0).sum()`
Out [11]:	`0.953333333333333`

In [12]:	`from sklearn.metrics import precision_score` `precision_score(y_true, y_pred, average="micro")`
Out [12]:	`0.9666666666666667`
In [13]:	`precision_score(y_true, y_pred, average="macro")`
Out [13]:	`0.9666219376328072`
In [14]:	`precision_score(y_true, y_pred, average=None)`

Out [14]:	`array([0.97916667, 0.98 , 0.98 ,`
	`0.94871795, 0.91111111, 0.97619048,`
	`0.97297297, 1. , 0.95652174,`
	`0.96153846])`

로지스틱 회귀_다중클래스 분류를 코드로 구현

- micro를 선택하면 전체 평균값, macro를 선택하면 각 라벨별 결과의 합에 대한 평균을 나타냄

In [11]:	`result = confusion_matrix(y_true, y_pred)` `result.diagonal().sum() / result.sum(axis=0).sum()`
Out [11]:	`0.953333333333333`

In [12]:	`from sklearn.metrics import precision_score` `precision_score(y_true, y_pred, average="micro")`
Out [12]:	`0.9666666666666667`
In [13]:	`precision_score(y_true, y_pred, average="macro")`
Out [13]:	`0.9666219376328072`
In [14]:	`precision_score(y_true, y_pred, average=None)`

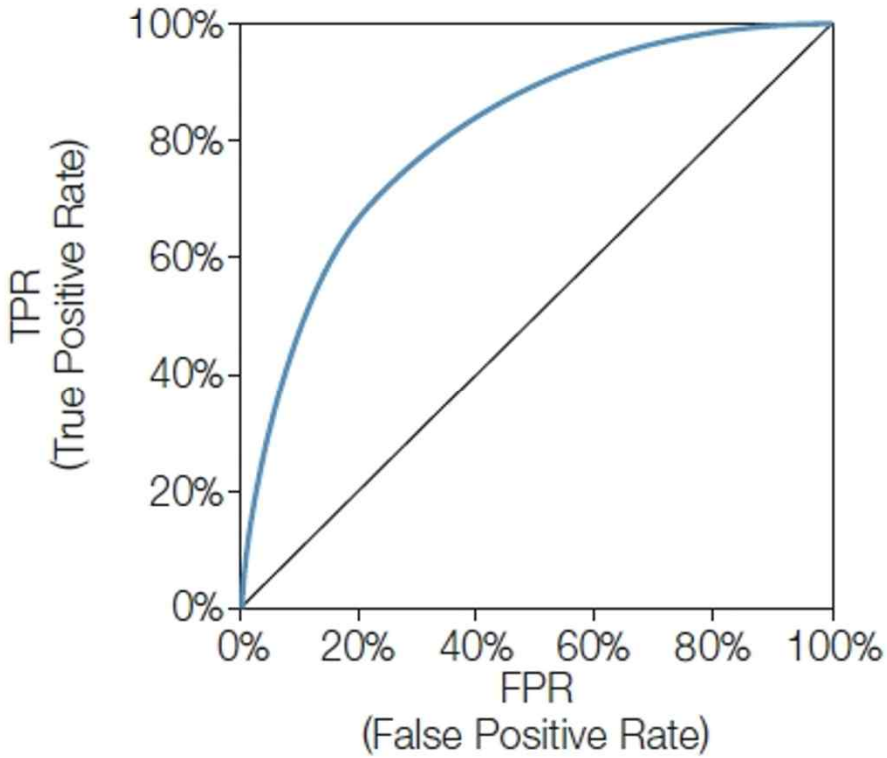
Out [14]:	`array([0.97916667, 0.98 , 0.98 ,`
	`0.94871795, 0.91111111, 0.97619048,`
	`0.97297297, 1. , 0.95652174,`
	`0.96153846])`

정밀도와 민감도의 트레이드오프

- 정밀도(precision)와 민감도(recall)는 일반적으로 둘 다 동시에 상승하기 어렵고 임계값(threshold)에 따라 변화가 일어남
  - 두 값을 모두 고려하여 성능을 측정하기 쉽지 않음
- ROC 커브(ROC curve) : 분류기의 임계값을 지속적으로 조정하여 정밀도와 민감도 간의 비율을 도식화
  - 'Receiver Operating Characteristics'의 약자
  - 클래스의 예측 확률이 나오는 모델에 사용 가능

ROC 커브 표현하기

- TPR(True Positive Rate)과 FPR(False Positive Rate)을 각각 y축, x축에 나타내어 그래프를 작성



데이터	클래스	임계값
1	P	0.9
2	P	0.8
3	N	0.7
4	P	0.6
5	P	0.55
6	N	0.54
7	N	0.53
8	N	0.51
9	P	0.5
10	N	0.4

그림1. 일반적인 ROC 커브와 각 데이터의 예측 확률

로지스틱 회귀_ROC 커브와 AUC

표1. 일반적인 ROC 커브와 각 데이터의 예측 확률

데이터	클래스	임계값	TP	FP	TN	FN	TPR	FPR
1	P	0.9	1	0	5	4	0.2	0
2	P	0.8	2	0	5	3	0.4	0
3	N	0.7	2	1	4	3	0.4	0.2
4	P	0.6	3	1	4	2	0.6	0.2
5	P	0.55	4	1	4	1	0.8	0.2
6	N	0.54	4	2	3	1	0.8	0.4
7	N	0.53	4	3	2	1	0.8	0.6
8	N	0.51	4	4	1	1	0.8	0.8
9	P	0.5	5	4	0	1	1	1
10	N	0.4	5	5	0	0	1	1

로지스틱 회귀_ROC 커브와 AUC

- TPR과 FPR의 값을 연결하여 그래프를 작성

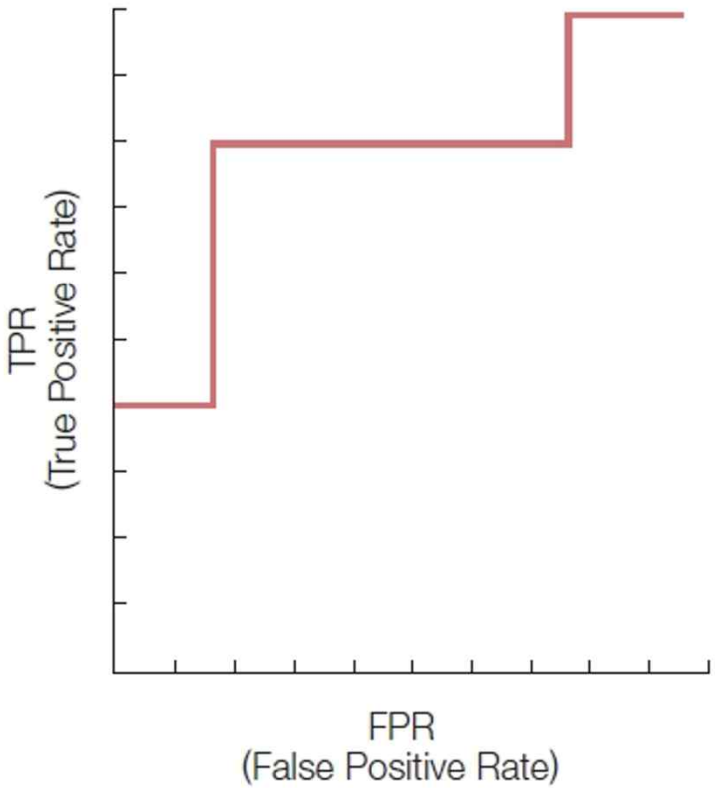


그림 2. 표3을 기준으로 한 ROC 커브

로지스틱 회귀_ROC 커브와 AUC

- AUC(Area Under Curve) : ROC 커브 하단의 넓이

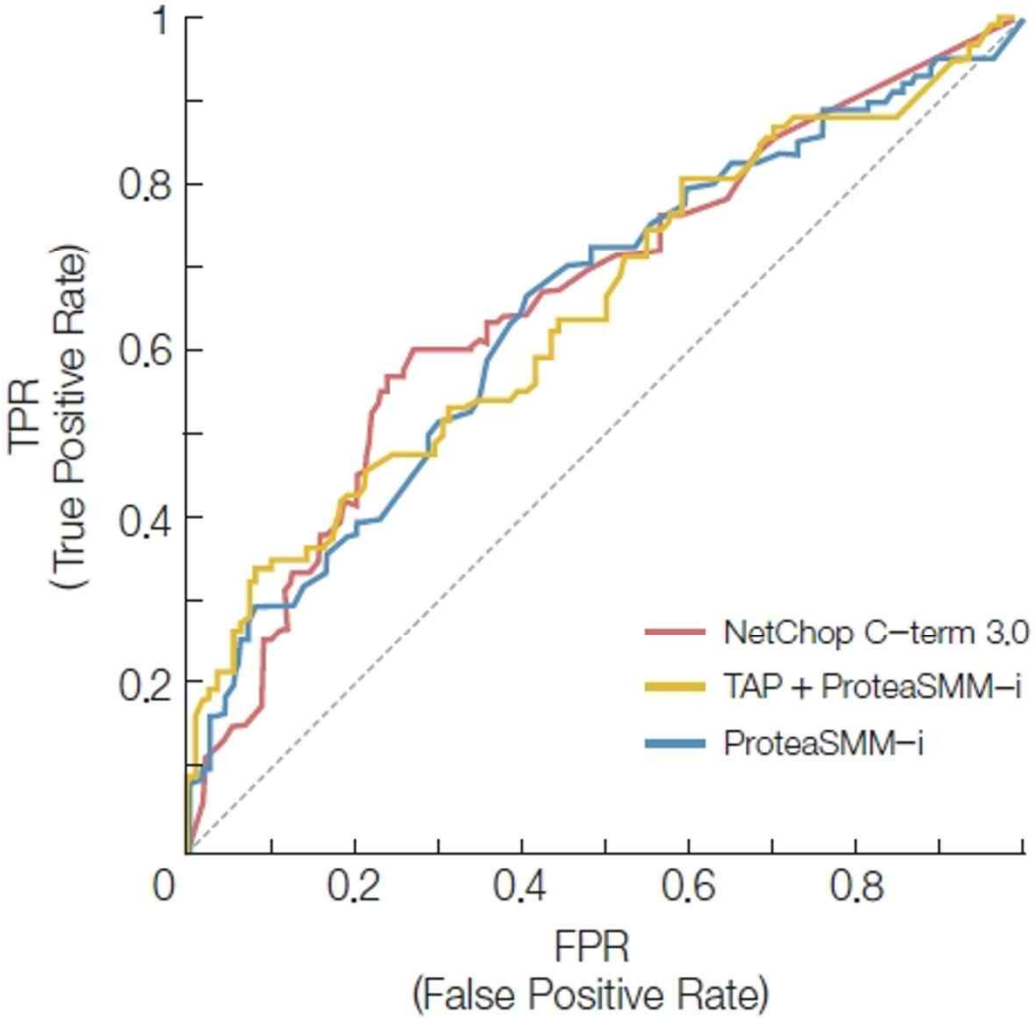


그림 3. 여러 모델의 성능을 상대적으로 비교하기 위한 그래프

- 대표적인 ROC 커브로 모델들의 성능을 단 하나의 숫자로 표현할 수 있다는 점에서 불균형 데이터셋 (imbalanced dataset)의 성능을 평가할 때 많이 사용

ROC 커브와 AUC를 사이킷런 코드로 구현하기

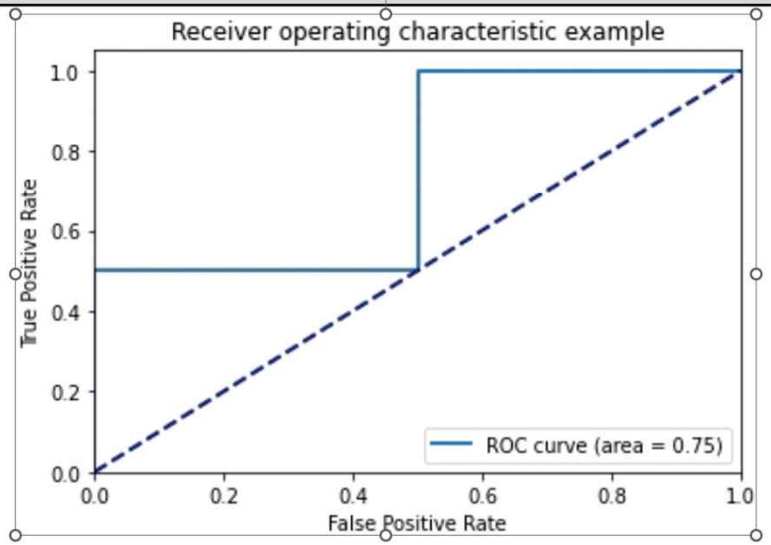
- 정답 y 값과 각 항목별 예측 확률을 scores에 저장
- ROC 커브 함수인 roc_curve로 fpr, tpr, thresholds 반환

In [1]:
```
import numpy as np
from sklearn import metrics

y = np.array([1, 1, 2, 2])
scores = np.array([0.1, 0.4, 0.35, 0.8])
fpr, tpr, thresholds = metrics.roc_curve(y, scores,
pos_label=2)

# fpr - array([0. , 0. , 0.5, 0.5, 1. ])
# tpr - array([0. , 0.5, 0.5, 1. , 1. ])
# thresholds - array([1.8 , 0.8 , 0.4 , 0.35, 0.1 ])
```

로지스틱 회귀_ROC 커브와 AUC

In [2]:	y = np.array([1, 1, 2, 2]) pred = np.array([0.1, 0.4, 0.35, 0.8]) fpr, tpr, thresholds = metrics.roc_curve(y, pred, pos_label=2) roc_auc = metrics.auc(fpr, tpr) roc_auc
Out [2]:	0.75
In [3]:	import matplotlib.pyplot as plt plt.figure() lw = 2 plt.plot(fpr, tpr, lw=lw, label='ROC curve (area = %0.2f)' % roc_auc)

로지스틱 회귀_ROC 커브와 AUC

```
plt.plot([0, 1], [0, 1], color='navy', lw=lw,
linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic
example')
plt.legend(loc="lower right")
plt.show()
```

Out [3]: